

Memristor-Inspired Neuromorphic Control for Robotics:

A Comparative Study of SNN and DNN Architectures with FPGA-Accelerated Synaptic Emulation

Undergraduate Thesis Proposal — Electrical & Computer Engineering

Abstract

This thesis proposes a hardware-software co-design framework for neuromorphic robotic control, combining spiking neural networks (SNNs) with memristor-inspired synaptic plasticity and FPGA-accelerated weight update logic. A leaky integrate-and-fire (LIF) SNN with spike-timing-dependent plasticity (STDP) is deployed in a simulated obstacle-avoidance task (PyBullet) and evaluated against a trained deep neural network (DNN) baseline and a reinforcement-learning (RL) policy. The FPGA implements the parallel synaptic conductance update array in hardware, directly emulating the crossbar-array dynamics of physical memristive devices. The central hypothesis is that online synaptic adaptation—impossible in a frozen DNN—enables measurably faster performance recovery when the environment changes mid-episode, while the FPGA acceleration provides latency and throughput advantages that justify the hardware-software co-design perspective. Metrics include response latency, synaptic operation count (SynOps), adaptation recovery time, and robustness under sensor noise.

1. Motivation and Research Gap

Modern robotic controllers are overwhelmingly based on deep neural networks trained offline: once deployed, their weights are frozen. This works well in static environments but introduces a fundamental brittleness—any distributional shift (new obstacles, sensor degradation, lighting changes) requires costly retraining. Biological neural systems solve this through *online synaptic plasticity*: the synapse itself carries memory and continues adapting throughout the lifetime of the organism.

Memristive devices are the closest physical analog to biological synapses. Their resistance state encodes a weight, updates depend on the history of current flow (state retention), the relationship is nonlinear and bounded (saturation), and device fabrication introduces stochasticity—all hallmarks of biological synaptic behavior [1]. An FPGA crossbar array is the most accessible way to emulate this parallel, in-memory computation without physical memristor hardware.

Core Research Questions

- RQ1.** Does an SNN with online STDP plasticity recover navigation performance significantly faster than a frozen DNN when the environment changes mid-episode?
- RQ2.** Does FPGA-parallel synaptic update logic reduce total weight-update latency compared to sequential CPU/GPU computation, and by how much?
- RQ3.** What is the energy-efficiency tradeoff (SynOps vs. multiply-accumulate operations) between the SNN and DNN approaches at equal task performance?

2. Background

2.1. Memristive Devices and Synaptic Emulation

A memristor is a two-terminal passive element whose resistance $R(t)$ is a function of the history of current through it [1]. Its conductance $g = 1/R$ evolves as:

$$\frac{dg}{dt} = f(g, V) = \alpha V(t) [g_{\max} - g] \cdot \mathbf{1}_{V>0} - \beta V(t) [g - g_{\min}] \cdot \mathbf{1}_{V<0} \quad (1)$$

where α, β are potentiation/depression rates, and $[g_{\min}, g_{\max}]$ bounds the conductance. This naturally implements bounded, nonlinear, state-dependent weight updates—exactly the properties we model in software and emulate on FPGA.

2.2. Leaky Integrate-and-Fire Neuron

Each neuron in the SNN follows the LIF model:

$$\tau_m \frac{dV_{\text{mem}}(t)}{dt} = -V_{\text{mem}}(t) + R_m I_{\text{syn}}(t) \quad (2)$$

where τ_m is the membrane time constant, R_m is membrane resistance, and I_{syn} is the total synaptic input current. A spike is emitted and the membrane is reset when $V_{\text{mem}}(t) \geq V_{\text{th}}$:

$$s(t) = \begin{cases} 1 & \text{if } V_{\text{mem}}(t) \geq V_{\text{th}} \\ 0 & \text{otherwise} \end{cases}, \quad V_{\text{mem}} \leftarrow V_{\text{reset}} \text{ after spike.} \quad (3)$$

The synaptic input is filtered through an exponential kernel:

$$I_{\text{syn}}(t) = \sum_j w_j \sum_{t_j^f} \exp\left(-\frac{t - t_j^f}{\tau_s}\right) \quad (4)$$

where t_j^f are the firing times of presynaptic neuron j and w_j is its synaptic weight.

2.3. Spike-Timing-Dependent Plasticity (STDP)

STDP is the biologically observed learning rule in which synaptic strength is modulated by the relative timing of pre- and post-synaptic spikes [2]. The weight update for synapse j is:

$$\Delta w_j = \begin{cases} A_+ \exp\left(-\frac{\Delta t}{\tau_+}\right) & \text{if } \Delta t > 0 \quad (\text{pre before post: LTP}) \\ -A_- \exp\left(\frac{\Delta t}{\tau_-}\right) & \text{if } \Delta t < 0 \quad (\text{post before pre: LTD}) \end{cases} \quad (5)$$

where $\Delta t = t_{\text{post}} - t_{\text{pre}}$, and A_+, A_-, τ_+, τ_- are positive constants controlling potentiation and depression amplitudes and time windows. Weights are bounded: $w_j \in [w_{\min}, w_{\max}]$, linking directly to the memristor conductance model in Eq. (1).

Link to memristor: Eq. (5) is a discretized, spike-triggered version of Eq. (1). A positive voltage pulse (pre fires before post) increases conductance (LTP); a negative pulse does the reverse (LTD). This is why the FPGA implementation of the weight-update crossbar is simultaneously a memristor emulator and an STDP accelerator.

3. System Architecture

The full system has three layers: simulation environment, neural controller, and FPGA co-processor.

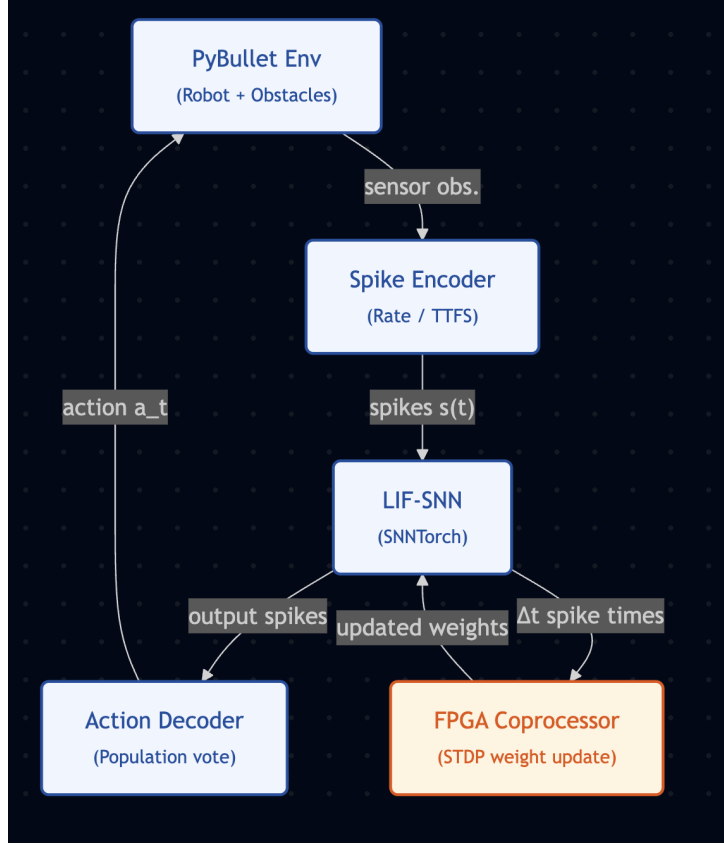


Figure 1: System Architecture showing the closed-loop control and FPGA co-processor.

3.1. Baseline Controllers (Comparison Group)

Three baselines are used to establish a fair and rigorous comparison:

1. **MLP-DNN (frozen):** A multi-layer perceptron trained offline via supervised imitation learning on expert trajectories. After training, weights are fixed. This isolates the effect of online plasticity.
2. **MLP-DNN (Online-Updating):** An MLP that performs continuous online learning via standard stochastic gradient descent (SGD) on a sliding window of recent observations. This isolates the true variable: neuromorphic STDP plasticity versus conventional continuous deep learning.
3. **RL Policy (PPO):** A proximal policy optimization agent with an MLP policy network, trained in PyBullet. This is the strongest baseline and represents the current state of practice for learned robotic control.

Why not CNN? CNNs are designed for grid-structured spatial data (images). The navigation task uses LiDAR distance readings and pose vectors—a flat, non-spatial observation space better suited to an MLP. Using CNN as the baseline would be testing the wrong inductive bias, not the right one.

3.2. Proposed Controller: LIF-SNN with STDP

1. **Input encoding:** LiDAR readings and velocity are encoded as spike trains using rate coding (firing rate proportional to stimulus intensity) for distance channels and time-to-first-spike (TTFS) coding for urgency signals.

2. **Network:** A fully connected LIF-SNN (3–4 layers, ~ 512 – 1024 neurons) implemented in SNN-Torch [7]. Initial weights are pretrained via surrogate-gradient backpropagation to match the MLP-DNN, then *released* to online STDP adaptation during evaluation.
3. **Output decoding:** Population vote over output neurons, one population per discrete action (left, right, forward, brake).
4. **Online adaptation:** At each timestep, spike times are sent to the FPGA, which computes Δw_{ij} from Eq. (5) in parallel across all synapses and returns updated weights before the next action.

3.3. FPGA Synaptic Update Coprocessor

The key claim of this thesis is that the STDP weight-update step is structurally identical to memristor crossbar operation, and that its parallelism can be hardware-exploited. On a crossbar, $N \times M$ synapses update *simultaneously* in $O(1)$ time. On a CPU, this is an $O(N \cdot M)$ sequential loop.

The FPGA implementation (targeting a Xilinx Zynq-7000 or Intel Cyclone V) consists of:

- **Spike-time registers:** One register per neuron, holding the most recent spike timestamp t_j^f in fixed-point format (16-bit, 0.1 ms resolution).
- **Δt computation array:** $N \times M$ parallel subtractors computing $\Delta t_{ij} = t_i^{\text{post}} - t_j^{\text{pre}}$ for every synapse simultaneously.
- **Exponential lookup table (LUT):** Pre-computed STDP kernels stored in BRAM; index by $|\Delta t|$ quantized to 64 bins.
- **Weight update array:** $N \times M$ parallel multiply-accumulators applying the LTP/LTD rule and clamping to $[w_{\min}, w_{\max}]$.
- **AXI4-Stream (DMA) interface:** Implements Direct Memory Access for burst-transferring entire vectors of spike states directly from CPU RAM to the FPGA, preventing the I/O bottlenecks inherent to single-register AXI4-Lite transfers.

This architecture directly emulates a memristive crossbar: each FPGA multiplier corresponds to one synapse, spike arrival is the voltage pulse, and the weight register is the conductance state g . The difference from Eq. (1) is that we operate in discrete time on spike events rather than continuous voltage.

4. Experimental Setup

4.1. Simulation Environment: PyBullet

All experiments run in PyBullet [12], a real-time physics simulation library. The scenario:

- **Robot:** Differential-drive ground vehicle with 360° LiDAR (32 beams, 8 m range), forward-facing depth camera, and IMU.
- **Environment:** Enclosed arena (10×10 m) with randomized static obstacles (boxes, cylinders) and dynamic obstacles (moving at 0.3–1.0 m/s).
- **Task:** Reach a goal position without collision. Episode length: 60 s.
- **Environment shift:** At $t = 30$ s, the obstacle density doubles and three new dynamic obstacles are injected. This is the critical test of adaptivity.

4.2. The Key Experiment: Online Adaptation Under Distribution Shift

This is the experiment that differentiates this thesis from prior work. The protocol:

1. Run all controllers (MLP-DNN frozen, MLP-DNN online, PPO, SNN-STDP) for the first 30 s with a fixed environment (Phase 1).
2. At $t = 30$ s, inject the environment shift without stopping the episode (Phase 2).
3. Measure **performance recovery time**: how many seconds until each controller’s collision-free navigation rate returns to within 10% of its Phase 1 baseline.
4. The MLP-DNN (frozen) *cannot* adapt. The PPO policy also cannot adapt online (its policy is frozen post-training). The evaluation focuses on comparing the recovery times between the SNN-STDP and the MLP-DNN (online) controllers.

4.3. Evaluation Metrics

Metric	Definition	Unit
Response latency	$T_{\text{resp}} = T_{\text{action}} - T_{\text{sensor}}$	ms
Energy efficiency	Synaptic operations: $\text{SynOps} = \sum_t \sum_{ij} s_i(t) \cdot \mathbf{1}_{w_{ij} \neq 0}$	ops/step
Adaptation recovery	τ_{rec} : seconds to recover 90% task performance after shift	s
Robustness	Success rate under additive Gaussian noise $\sigma \in \{0, 0.1, 0.3, 0.5\}$	%
FPGA speedup	$S = T_{\text{CPU}}/T_{\text{FPGA}}$ for the synaptic weight-update step	$\times$

Why SynOps and not $\sum |\Delta w|$? SynOps counts how many multiply-accumulate operations the network actually performs per timestep, accounting for spike sparsity. It is the standard neuromorphic efficiency metric [5] and maps directly to hardware energy because neurons that do not spike consume no compute. $\sum |\Delta w|$ has no physical interpretation and cannot be compared across architectures.

4.4. Software Stack

- **SNN framework:** SNN Torch [7] (Python, PyTorch backend)
- **RL baseline:** Stable-Baselines3 (PPO) [11]
- **Simulation:** PyBullet [12]
- **FPGA:** Xilinx Vivado / Intel Quartus (target: Zynq-7020 or Cyclone V)
- **Hardware interface:** PyNQ (Python-to-FPGA AXI DMA bridge)
- **Logging:** Weights & Biases for experiment tracking

5. Memristor Emulation Fidelity

To ground the software and FPGA models in physical device behavior, the Biolek memristor model [6] is used as a reference:

$$\frac{dx}{dt} = \mu_v \frac{R_{\text{on}}}{D^2} i(t) f(x) \quad (6)$$

where $x \in [0, 1]$ is the normalized state variable (maps to normalized conductance g), μ_v is the ion mobility, D is the device length, and $f(x)$ is a window function that prevents state from leaving $[0, 1]$. The STDP rule in Eq. (5) is a spike-triggered, discrete-time approximation of Eq. (6): a positive Δt corresponds to a net positive charge transfer (potentiation pulse), and a negative Δt to a depression pulse.

The three claimed memristive properties are emulated as follows:

Memristor Property	Software Model	FPGA Emulation
State retention	Weight registers persist between steps	BRAM weight array
Nonlinearity & saturation	$w \in [w_{\min}, w_{\max}]$ clamp	Saturating adder
Stochasticity	Additive noise $\epsilon \sim \mathcal{N}(0, \sigma_w^2)$	LFSR noise unit

6. Expected Results and Hypotheses

Hypotheses

- H1. Adaptation:** The SNN-STDP controller will recover 90% task performance after the environment shift within $\tau_{\text{rec}} < 5$ s, outperforming both the frozen MLP-DNN and the online-updating MLP-DNN.
- H2. Energy:** SNN-STDP SynOps will be 3–10 $\times$ lower than DNN multiply-accumulate operations at equal pre-shift performance, due to spike sparsity.
- H3. Latency:** FPGA parallel weight update via AXI-DMA will be 10–50 $\times$ faster than CPU sequential computation for the weight-update step alone (network size dependent).
- H4. Robustness:** SNN success rate will degrade more gracefully under increasing sensor noise σ , due to the temporal integration inherent in LIF dynamics acting as a low-pass filter.

The hypotheses are intentionally falsifiable. If H1 does not hold, the thesis conclusion is that conventional DNNs + fast online retraining is a better practical strategy than online STDP plasticity for this task class—which is itself a valuable finding.

7. Project Timeline and Budget

7.1. Phased Timeline (6–9 months)

Phase	Duration	Deliverables
1	Weeks 1–4	Literature review finalized; PyBullet environment built; MLP-DNN baselines (frozen and online) trained and evaluated
2	Weeks 5–10	SNN implemented in SNN Torch; surrogate-gradient pretraining; initial STDP adaptation experiments
3	Weeks 11–16	PPO baseline trained; full comparative experiment across all metrics; environment-shift protocol run
4	Weeks 17–22	FPGA STDP accelerator designed and synthesized; hardware-in-loop timing measurements; FPGA vs. CPU speedup characterization
5	Weeks 23–28	Thesis writing; figure generation; revision; presentation prep

7.2. Budget Estimate

Item	Cost (EUR)	Notes
Xilinx Zynq-7020 SoC board (e.g. Digilent Arty Z7-20)	180–220	AXI + ARM core; PyNQ compatible
<i>Alternative:</i> Intel Cyclone V (DE10-Nano)	120–150	Budget option; HPS for Python side
LiDAR sensor (RPLIDAR A1M8, if physical demo)	100–130	Optional; simulation covers this
Differential-drive robot chassis (if physical demo)	80–120	e.g. Waveshare RaspiCar or similar
GPU cloud compute (Vast.ai, 50 h at ~€1.2/h)	60–80	For PPO training and SNN pretraining
Miscellaneous (cables, SD cards, USB hubs)	20–30	
Total (simulation-only + FPGA)	260–330	
Total (FPGA + physical robot)	440–580	

Both scenarios are well within a 1000–1500 EUR budget, leaving substantial headroom.

8. Related Work and Further Reading

The following papers and resources define the landscape this thesis operates within. They are grouped by sub-topic for targeted reading.

Memristive Devices and Neuromorphic Hardware

- Strukov et al. (2008), *Nature*—“The missing memristor found.” Foundational paper establishing memristors as circuit elements [1].
- Biolek et al. (2009)—SPICE memristor model with window functions [6].
- Merolla et al. (2014), *Science*—IBM TrueNorth: 4096 cores, 1 million neurons, 256 million synapses at 70 mW [4].
- Davies et al. (2018), *IEEE Micro*—Intel Loihi: on-chip STDP, first programmable neuromorphic chip with online learning [5].

- Shooshtari et al. (2025), *Advanced Intelligent Systems*—Review of memristors as the hardware bridge between in-memory computing and SNNs.

Spiking Neural Networks: Theory and Training

- Maass (1997), *Neural Networks*—Networks of spiking neurons as the third generation of neural network models [3].
- Bi & Poo (1998), *J. Neuroscience*—Original STDP experimental measurement in hippocampal neurons [2].
- Lee et al. (2016), *Front. Neuroscience*—Training deep SNNs via surrogate gradients; the method used for SNN pretraining here.
- Eshraghian et al. (2023), *Proc. IEEE*—SNN-Torch framework paper; the primary software tool for this thesis [7].

Neuromorphic Robotics: Most Directly Relevant

- Lobov et al. (2020), *Front. Neuroscience*—STDP-controlled mobile robot with memristive devices; nearest predecessor to this work [8].
- Juarez-Lora et al. (2022), *Front. Neurorobotics*—R-STDP for robotic arm control under changing joint friction; demonstrates online adaptation [9].
- Abdelrahman et al. (2024/2025), *Int. J. Robotics Research*—Convolutional SNN for obstacle avoidance on a Kinova Gen3 arm; tested in Gazebo and real world.
- Stroobants et al. (2024), *arXiv:2411.13945*—First full neuromorphic attitude control SNN deployed on a real Crazyflie quadrotor.
- Supic & Stewart (2022), *ACM*—SNN adaptive control for robot arms; 15% faster task completion with online learning over fixed policies.

FPGA Neuromorphic Acceleration

- Qiao et al. (2015), *Front. Neuroscience*—ROLLS neuromorphic chip with embedded FPGA interface for spike routing; architectural reference [10].
- Neil & Liu (2014), *IEEE BioCAS*—Minitaur: FPGA-based SNN simulator with on-chip STDP; close analog to the FPGA module proposed here.
- Prajapati et al. (2025), *arXiv:2507.20998*—Memristive SNN with supervised in-situ STDP; most recent circuit-level implementation reference.

Reinforcement Learning Baseline

- Schulman et al. (2017), *arXiv:1707.06347*—PPO algorithm used for the RL baseline policy.
- Raffin et al. (2021), *JMLR*—Stable-Baselines3 library [11].

9. Conclusion

This thesis proposes a concrete, falsifiable comparative study of neuromorphic vs. conventional robotic control, grounded in three interlocking contributions: (1) a software SNN with biologically rigorous STDP demonstrating online adaptivity under environment shift, (2) an FPGA coprocessor that is simultaneously a memristive crossbar emulator and a synaptic update accelerator, and

(3) a quantitative evaluation framework using standard metrics (SynOps, recovery time, FPGA speedup) that allows direct comparison with the existing literature.

The project is achievable within a 6–9 month timeline and a budget of 440–580 EUR including physical hardware, or under 350 EUR for a simulation-only study. The FPGA component is not decorative—it is the hardware argument for why memristor-inspired computation is worth building: it makes the parallel crossbar structure of synaptic learning physically instantiable without fabricating nanoscale devices.

References

- [1] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, “The missing memristor found,” *Nature*, vol. 453, pp. 80–83, 2008.
- [2] G. Q. Bi and M. M. Poo, “Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type,” *Journal of Neuroscience*, vol. 18, no. 24, pp. 10464–10472, 1998.
- [3] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [4] P. A. Merolla *et al.*, “A million spiking-neuron integrated circuit with a scalable communication network and interface,” *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [5] M. Davies *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [6] Z. Biolek, D. Biolek, and V. Biolkova, “SPICE model of memristor with nonlinear dopant drift,” *Radioengineering*, vol. 18, no. 2, pp. 210–214, 2009.
- [7] J. K. Eshraghian *et al.*, “SNN Torch: Accelerating neuromorphic engineering with a deep learning algorithm library,” *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1–19, 2023.
- [8] S. A. Lobov *et al.*, “Spatial properties of STDP in a self-learning spiking neural network enable controlling a mobile robot,” *Frontiers in Neuroscience*, vol. 14, p. 88, 2020.
- [9] A. Juarez-Lora *et al.*, “R-STDP spiking neural network architecture for motion control on a changing friction joint robotic arm,” *Frontiers in Neurorobotics*, vol. 16, 2022.
- [10] N. Qiao *et al.*, “A reconfigurable on-line learning spiking neuromorphic processor comprising 256 neurons and 128K synapses,” *Frontiers in Neuroscience*, vol. 9, p. 141, 2015.
- [11] A. Raffin *et al.*, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [12] E. Coumans and Y. Bai, “PyBullet, a Python module for physics simulation for games, robotics and machine learning,” <http://pybullet.org>, 2016–2021.